Ideas

- ~~Job/Internship Automation Agent~~
- App,
-

branc

Prev ideas

- https://la-hacks-2025.devpost.com/project-gallery
- https://la-hacks-2024.devpost.com/project-gallery

Github

- https://github.com/cr0w1234/LAHacks2026

Let me pull from more specific sources — Reddit threads and product hunt to get developer and user-focused pain points.Solid data pulled. Let me synthesize what I found into the actual underserved pain points — the ones that come up repeatedly, don't have clean solutions, and where AI agents (specifically, not just "AI apps") could genuinely change the game.

The meta-insight from the research

Most existing AI tools *add* decisions rather than remove them. They surface more options faster, but the human still has to pick, execute, and follow up. The pain points that keep going unsolved share a common shape: **they require real-world action across multiple systems, coordination with other humans, or closing a loop that spans days/weeks.**That's exactly the gap agents fill.

There's even a term emerging for this now — "AI decision fatigue" — where the mental load doesn't disappear when you use AI, it just shifts into a supervisor role. The opportunity is agents that actually *close the loop* without asking you 12 follow-up questions.

WORK

The email triage gap (still not solved). Workers get ~120 emails/day, 83% report work-related stress, 50% feel obligated to reply within an hour. Existing tools (filters, labels, Gmail's AI categories) *organize* mail but don't *respond*. What's missing: an agent that drafts responses in your voice for routine emails, flags only ones where your judgment is actually needed, and handles the entire back-and-forth for scheduling, package returns, vendor admin, etc. Superhuman and Shortwave try but stop at drafting.

Meeting → action item → actually done. Every company complains: meetings happen, AI notes get taken, nothing happens. Otter/Granola/Fireflies generate summaries nobody reads. The unsolved piece is an agent that takes meeting outputs and actually *creates* the Jira tickets, drafts the follow-up emails, books the next meeting, nudges the stakeholders. The bridge between "meeting happened" and "work got done" is still entirely manual.

The "50% of employees don't know what's expected of them" problem. Remote work research keeps surfacing this. Status updates are performative; nobody reads them. An agent that synthesizes "what my team is actually working on" from commits, tickets, Slack, and docs — without requiring anyone to write status updates — would be huge. Nobody has cracked this because it requires reading across 5+ tools.

The "where did that thing go" search gap. Knowledge workers spend a real chunk of time hunting for a doc/message/decision across Slack + Notion + Gmail + Drive. Universal search tools (Glean, etc.) exist but cost enterprise money. For individuals, nothing.

CONNECT (the richest vein for a hackathon)

The "group chat dies" problem. Every adult has 3–5 group chats with close friends that slowly go dormant. Classic diffusion-of-responsibility: everyone assumes someone else will suggest the next hang. Research backs this up — adults need "continuous unplanned interaction" to maintain friendship but have fewer environments producing it. **No AI tool addresses this.** An agent that notices a group hasn't met in 6 weeks, knows everyone's calendars, and proactively proposes 3 concrete dates + activity options is a real product nobody has built.

Scheduling with friends is "a diplomatic summit." That's a direct quote from the friendship research. Calendly works for business, but nobody sends a Calendly link to their friends. Doodle polls die with zero responses. The unsolved pain: an agent that handles the multi-party scheduling *conversation* with humor and social intelligence, not a formal tool.

The loneliness epidemic among college students. 52% of college students report high loneliness in 2025. Students know they should go to events, join clubs — they don't, because of what researchers call "covert avoidance" (showing up but staying on your phone). The agent opportunity: not another Meetup clone, but a personal "social concierge" that knows you, your friends, what events are happening, and actively proposes "hey, Sarah who you met at that hackathon is going to this thing tonight, want to go together?" — closing the loop on weak ties.

Reconnecting lost friends. Psychology Today's research emphasizes "weak ties" as the overlooked friendship goldmine. Nobody has time to manually scan their contacts and think "who haven't I talked to in a year that I should?" An agent that monitors your relationship decay — last contact date, life events from LinkedIn/Instagram — and prompts meaningful reconnections is deeply underserved.

Friend matchmaking within your network. When you meet someone new and realize "oh, you'd love my friend Jamie" — but nobody ever follows through on the intro. An agent that knows your social graph and actively suggests/executes intros is novel.

Long-distance friendship maintenance. When college friends scatter, the relationship decays not because of intent but because of coordination cost. Timezone-aware agents that find a 20-minute window across 4 people's calendars are unsolved.

PLAY

Dinner paralysis. 77% of Americans are too exhausted to cook after work; 68% say deciding what to eat is their biggest mealtime challenge. Current tools (Yummly, etc.) suggest recipes but don't integrate *your fridge + your calendar + your mood + your grocery store*. An agent that says "you have chicken thighs from Tuesday that need using, you're stressed because you have a 3pm meeting tomorrow, here's the 20-minute thing" — that's a real pain killer.

"What to watch" decision fatigue. Research literally measures this as a decision-fatigue task. Netflix's recommender optimizes engagement, not "help the user actually pick." An agent that asks you how you feel in 3 words and picks *for you* (executes the play command) is a different product.

The hobby beginner's cliff. Motivation research shows people quit new skills at the first real error. "Give yourself permission to be bad at it" is the advice, but the real gap is a personal coach that scales the right-sized challenge, notices when you're stuck, and actually intervenes. Duolingo does it for languages; nothing does it for guitar, drawing, cooking, coding.

Event discovery is still broken. Eventbrite, Partiful, Luma, Meetup, university mailing lists, Instagram stories — events live everywhere. Nobody aggregates them by "what my specific friend group would like this Saturday." Especially for students, where campus events are scattered across 15 channels.

Finding a specific niche community. The research says hobbies work best as shared commitments. Finding the 3 other people in LA who want to do Rust pair-programming on Saturdays is currently impossible without manual Reddit/Discord trawling. An agent that goes *finds* those people and assembles the group is novel.

The pain points that are particularly AI-agent-shaped

Looking across all of the above, the common structure of "this needs agents, not apps" is:

1. **Requires action in multiple systems** (calendar + email + maps + payment)
2. **Requires coordination with other humans** (not just the user)
3. **Requires persistence over days/weeks** (not a single session)
4. **Involves "soft" judgment** that needs an LLM, not rules
5. **User doesn't want to micro-manage** — they want the outcome, not the choice

My top 5 picks for your hackathon

Ranked on novelty × demo-ability × "closes a loop an app can't":

1. **Group chat revival agent** — monitors friend group inactivity, drafts proposals, handles the scheduling dance across N people's calendars, executes the booking. Hits the loneliness crisis and the scheduling pain simultaneously. Massive demo appeal — judges are adults with dormant group chats.
2. **Dormant-tie reconnection agent** — scans your contacts, flags people you should reconnect with, drafts the "hey it's been a while, saw you got promoted — coffee?" message in your voice, follows up if no reply. Backed by real psychological research on weak ties.
3. **Meeting-to-done agent** — takes a meeting transcript, actually creates the tickets, sends the follow-ups, books the next meeting. Demo: live Zoom call ends, agent starts working, 2 minutes later shows what it did. Very judge-friendly.

4. **"What should I eat tonight" agent** — integrates your fridge photo, your calendar stress level, your dietary prefs, your grocery delivery. Executes: adds to cart, or books a table, or DMs a friend to meet at a spot. The difference from Yummly is it *commits* to an answer.
5. **Personal social concierge** — knows your friends, knows what's happening this week, proactively surfaces "Alex is free Thursday and the band you both like is playing, I found two tickets, yes/no?" The Cursor-for-social-life.

All 5 are multi-agent natural fits (scheduler agent + drafter agent + calendar agent + etc.) — which is exactly what Fetch.ai wants to see, and all 5 map cleanly to Flicker to Flow's "transform friction into function."

Want me to go deep on one of these — user journey, agent architecture, and what the winning demo looks like on stage?

CLAUDE’S OPINION ON THE CHAT IDEA

This is a genuinely strong concept. The core instinct — "proactive, not reactive" — is the right wedge, and it's exactly the "multi-agent orchestration that turns user intent into real, executable outcomes" language from Fetch.ai's prompt. Let me give you honest feedback, because there are a few places this could slip on execution.

What's working

The "ambient agent in a group chat" framing is novel. Every existing AI assistant is 1:1 and reactive. A group-aware, listening, proactively-contributing agent is a genuinely different product shape. Judges haven't seen it 50 times.

The dormancy-detection piece solves a real documented pain. Research-backed. Great demo moment: show a group chat that's gone silent for 3 weeks, watch the agent revive it.

Multi-agent decomposition is natural. Calendar agent, interest agent, event-discovery agent, suggestion agent, scheduler agent — maps cleanly to Fetch.ai's rubric.

"Executable outcome" is clearly demonstrable. The bar isn't "suggest," it's "book the thing, add to everyone's calendar, send the confirmation." You can hit that.

Where I'd push back

The scope is ambitious for 36 hours. You've described at least five big surfaces: messaging app, calendar integration, interest inference, proactive event discovery, and scheduling. Any one of those is a weekend. Building all five means each one gets 20% of the attention it needs, and the demo feels flimsy. Ruthlessly cut.

"AI listening in the chat" has a creepiness problem. People hate the feeling of being surveilled by their group chat. The difference between "helpful butler" and "surveillance" is tone, consent, and when the agent speaks. If it chimes in on every message, people will mute it in a day. You need a deliberate spareness — the agent speaks only when it has something genuinely worth saying. This is a *product design* problem, not a technical one, and it's the thing that will make or break user love.

Privacy is a real judge question. "You're reading all our messages" will come up. Have a crisp answer: what's stored, what's ephemeral, what's processed locally vs sent to the LLM. Even a half-baked answer beats no answer.

Building a messaging app from scratch is a trap. You don't need to rebuild iMessage. The chat is a *container* for the demo. Use something you can stand up in 2 hours — a simple web chat, or even better, integrate into something existing (Slack, Discord, iMessage via a shortcut). Time spent on message delivery is time not spent on the agent logic, which is what judges actually score.

"Interest inference from chat history" is a rabbit hole. You could spend the whole hackathon on this and judges won't see it. Much safer: onboarding asks each user 5 interest questions explicitly. Infer more later if there's time.

Event discovery quality is a moat you won't build in 36 hours. Scraping Eventbrite, Partiful, Luma, Ticketmaster, local Instagram accounts — that's a real company. For the demo, curate a fixed list of 50 real upcoming LA events and let the agent match against those. Judges won't know or care. Full harness works later.

The version that actually wins

Cut aggressively. Here's the MVP I'd build:

Core loop, on stage:

1. Four pre-seeded users in a group chat (you + teammates, pre-populated interests).
2. Google Calendar integration — real, working, for those four accounts.
3. A chat interface (simple web app, left/right bubbles, one screen).
4. Someone types "we should hang soon" or the group has been silent for N days → agent chimes in with: "You four have all been free Saturday 2–6pm. There's a ramen festival in Little Tokyo Saturday 3pm, $15 tickets. Want me to book?" → someone taps yes → agent books, adds to 4 calendars, shows confirmation.

That's the hero demo. Everything else is garnish.

The agents (Fetch.ai rubric check):

- **Calendar agent** — reads everyone's free/busy, finds windows
- **Interest agent** — holds the group's interest profile
- **Event agent** — queries the curated event corpus (and a web search as a flourish)
- **Proposer agent** — the voice in the chat, composes the suggestion
- **Booking agent** — handles the "yes" → real-world action (Eventbrite API or just a mock confirmation email)

Five agents, each registered on Agentverse, each with a chat-protocol endpoint. Orchestrator routes. This hits every Fetch.ai rubric item cleanly.

The "proactive listening" angle, done right: not on every message. The agent listens for *triggers* — mentions of places ("Paris"), activities ("hiking"), time gaps ("haven't seen you in forever"), and silence (N days of no chat). Most of the time it shuts up. This restraint is what makes it feel magical rather than annoying.

Three strategic calls to make now

1. **Which one hero flow are you demoing?** Pick one. "Dormant group gets nudged into a Saturday plan" is probably the best (emotional, visible, clear outcome). Everything else supports that.
2. **Is your user "a whole friend group" or "one person with friends"?** The first sounds better but requires multiple accounts and adds massive friction to the demo. The second is easier and still powerful — the agent acts on your behalf to coordinate the group, even if only you are logged in. I'd strongly lean option two.
3. **What's the data source for events?** Decide Saturday morning and stop iterating on it. Eventbrite API is public and free. UCLA event calendar if you want local flavor. Don't burn hours here.

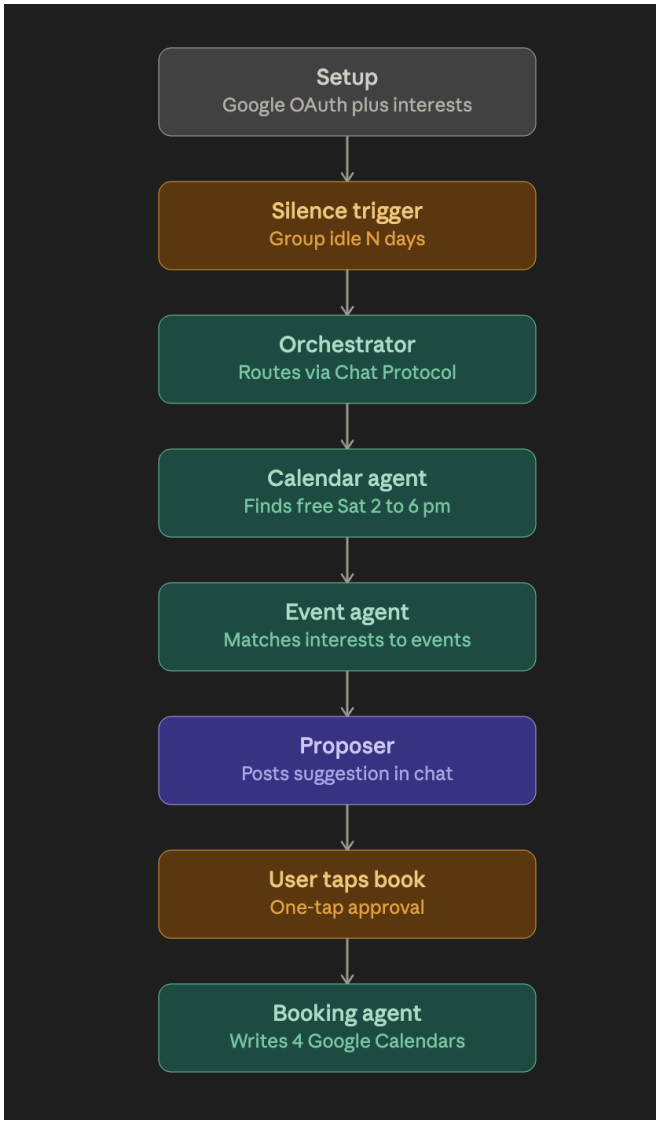
The pitch framing that wins

Don't pitch this as "an AI chat app." Pitch it as:

> "Your friend group has 40 shared messages a week and hasn't hung out in two months. Group chats are where plans go to die. We built an ambient agent that lives in your chat, knows everyone's calendars, and proposes real, executable plans — not suggestions, bookings. It's the friend who always has an idea and actually follows through."

That framing maps cleanly to Flicker-to-Flow ("turn friction into function"), to Fetch.ai ("executable outcomes"), and to a judge who has personally let a group chat die.

Bottom line: the idea is strong and on-rubric. The failure mode is scope, not concept. Cut to one hero flow, use a curated event list, skip building a real messenger, and demo the dormancy-revival moment live. Want me to sketch the specific agent architecture and what each agent's Chat Protocol message schema looks like?



The single most useful reframe: **all four deliverables serve one thing, and that thing is the demo video.** Judges at LA Hacks are watching a submission reel, not reading your repo. The Devpost description sets up the video, the video closes the sale, and the repo is a hygiene check that barely gets opened.

So the strategy is: **build backwards from a 90-second demo video.**

Here's what that looks like concretely.

Hour 0–2: Write the demo script before you write any code. Literally type out what the screen will show and what you'll say, second by second. If the script says "the agent posts a message saying 'you 4 are free Saturday 2–6pm, there's a ramen festival at 3, want me to book?'" — that tells you the exact copy the Proposer agent needs to produce, the exact event that needs to be in your JSON, and the exact fake calendar state your 4 seeded users need. The script is your product spec.

Hour 0–2 (in parallel): Register agents on Agentverse immediately. Not at hour 30. Agentverse registration + Chat Protocol wiring is the *prize qualifier* — if it's flaky at submission time you lose the $2.5K regardless of how good the product is. Get a stub agent registered and talking over Chat Protocol in the first few hours, then iterate.

Hour 2–20: Build the hero flow, nothing else. End-to-end stub by hour 4 per your own plan. Then flesh out each agent until the demo script runs cleanly. Every commit should be answering "does this make the video better?" If the answer is no, don't merge it.

Hour 20–28: Polish only what the camera sees. Phone-frame CSS, the message copy the Proposer produces, the confirmation screen, the pre-seeded avatars and names. Error handling that no one triggers in the demo is invisible work.

Hour 28–32: Record, write, submit. Record the demo video in multiple takes and pick the best one — budget 2 hours for this, it always takes longer than you think. Write the Devpost when you're fresh-ish, not at 4am. **Submit at hour 32**, not hour 36. A working submission at 32 with a buffer beats a better submission at 35:58 that fails to upload.

Hour 32–36: Buffer. Live pitch rehearsal, small fixes, anything that goes wrong. This time always disappears — that's why you submitted at 32.

A few rules that save hours:

- **Devpost is a marketing document, not a technical report.** Problem → product → 1 sentence on the tech → tracks → team. Write it in the Flicker to Flow voice ("friction into function") and the Fetch.ai voice ("executable outcomes"). Don't explain uAgents, judges know.
- **Repo needs a README, a one-command setup, and a GIF at the top.** That's it. No one is reading your source files.
- **Check the Figma Make Challenge box on Devpost** even if you barely used it for the mockup. Free swag prize.
- **If something breaks at hour 30, cut it, don't fix it.** Your demo script is the contract. If the Event agent's interest ranking gets weird, hardcode the Saturday ramen event for the demo and ship.

The failure mode to avoid: spending hour 20–28 making the booking agent robust to edge cases no judge will ever see, then recording the demo in one rushed take at 3am and submitting at 35:55. I'd bet half the teams this weekend do exactly that.

Want me to draft the 90-second demo script next? That's the highest-leverage hour-1 artifact and it'll tighten everything else.

LA Hacks 2026 — Group Chat Agent (Project Brief)

Shareable context doc. Paste into any AI chat to get useful input on this project.

The pitch

Group chats are where plans go to die. We're building an ambient AI agent that lives inside a friend group's chat, knows everyone's calendars, and proactively proposes real, executable plans — not suggestions, bookings. When a group has gone silent for weeks, or when someone mentions an interest in passing, the agent finds a time everyone's free, matches it to a real local event, and books it with one tap.

Framing: **"the friend who always has an idea and actually follows through."**

Hackathon context

LA Hacks 2026 — 36-hour hackathon at UCLA.

Tracks targeted (priority order)

1. **Flicker to Flow** (LA Hacks main track) — "turn friction into function" — directly applies to scheduling/social coordination friction.
2. **Fetch.ai — Agentverse: Search & Discovery of Agents** — $2,500 / $1,500 / $1,000. Requires agents registered on Agentverse, discoverable via ASI:One, implementing **Chat Protocol (mandatory)**. Payment Protocol optional.
3. **MLH add-ons** (easy wins — mostly checkbox):
    - Figma Make Challenge (check box on Devpost, show how Make fit into workflow)
    - Best Domain Name from GoDaddy Registry
    - Best Use of MongoDB Atlas (only if we actually use it)

Tech stack (locked)

- **Language:** Python
- **Agent framework:** uAgents (Fetch.ai's Python library) — required for Agentverse registration
- **Protocol:** Fetch.ai Chat Protocol (mandatory for the Fetch.ai prize)
- **LLM:** ASI:One for agent reasoning (preferred for the prize); Claude/GPT as fallback for tougher tasks
- **Calendar:** Google Calendar API (OAuth, read free/busy, write events)
- **Events data:** Curated JSON of ~50 real upcoming LA events. **No real scraper** — scope trap.

- **Frontend:** Web app rendered inside a **phone-shaped frame** for the demo (NOT a native mobile app — it just looks like a group chat on a phone for visual storytelling). React or plain HTML+JS.
- **Deployment:** Agents on Agentverse; frontend on Vercel or similar.
- **NOT using this weekend:** ElevenLabs/voice, voice messages, call-listening. Parked as post-hackathon directions.

MVP scope (ruthlessly locked — hero demo only)

1. 4 pre-seeded users in a group chat (team members with real Google accounts).
2. Group has been "silent" for N days (timestamp faked).
3. Agent proactively chimes in: *"You four are all free Saturday 2–6pm. There's a ramen festival in Little Tokyo Saturday 3pm, $15 tickets. Want me to book?"*
4. Someone taps yes.
5. Agent adds the event to all 4 Google Calendars and posts a confirmation.

Everything else is out of scope unless the hero flow is bulletproof first.

Agent architecture (5 agents on Agentverse)

- **Calendar agent** — reads free/busy across N users, finds overlap windows.
- **Interest agent** — holds each user's interest profile (from onboarding Q&A, not chat inference — scope cut).
- **Event agent** — queries curated event corpus, ranks by interest match.
- **Proposer agent** — composes the in-chat suggestion message in natural voice.
- **Booking agent** — executes the real-world action (Google Calendar writes, mock email confirmation).

An orchestrator routes between them via Chat Protocol. Each agent has its own README/keywords for Agentverse discoverability.

Strategic calls (decided)

- **User model:** single user acts on behalf of the group (not N logged-in users). Much simpler demo.
- **Listening model:** agent is triggered by specific signals (place names, activity words, N days of silence) — NOT every message. Restraint is the product.
- **Data source:** curated events JSON. Real scraping is post-hackathon.
- **Privacy answer for judges:** interests collected in onboarding (not inferred from messages); LLM calls are transient; data is per-group, not cross-group.

Team workflow

- Shared GitHub repo, short-lived feature branches, merge often.
- End-to-end skeleton (UI → orchestrator → stub agents → fake data) working by hour 4.
- Demo script drafted by hour 4, rehearsed by hour 30.
- Pitch deck started hour 4, finalized hour 32.
- Submission + demo video time-boxed and protected.

Open questions

- Final role split among teammates.
- Hosted Agentverse vs locally-run agents for the live demo.
- Specific event corpus — likely Eventbrite + UCLA events + a handful of LA venues, hand-picked.

Pitch framing (for judges)

- Tie back to **Flicker to Flow:** "friction into function" — group scheduling is pure friction; we turn it into a booked plan.
- Tie back to **Fetch.ai:** "executable outcomes" — our agents don't just suggest, they transact (calendar writes are real).
- The restraint angle is a feature, not a limitation: an AI that's quiet 99% of the time and perfect 1% of the time is the only one you'd actually let into your group chat.

How to give useful input on this project

- Be concise. Hackathon time is precious.
- Challenge scope creep aggressively. If a feature is proposed, ask what it replaces.
- Prioritize shipping over elegance. Ugly code that demos beats beautiful code that doesn't.
- Assume uAgents + Chat Protocol patterns when writing code.
- For architecture questions, 36-hour constraint first, scalability never.
- Flag anything that judges won't see in the demo — it's probably not worth doing.

User Stories

1. Account creation → profile, birthday etc → issue on how if users are already planning for a trip to somewhere, it’s kinda stupid to bring up someone’s birthday? Maybe if people are currently planning for a trip then don’t do any reminders? Think about this
2. Create a groupchat (invite members) - Trip Name
3. Option → Uploading Google Calender or manual

4. Auto-deletion of inactive chats → set expiration date 30,60,90,1 year
5. Sending a message
6. Right panel (swipe right for a list of ideas that have been talked about) - organizes all the events that have been brought up in chronological order) → say some like how many days until trip etc
7.
    a. hidden panel (all button) that brings up events that are not serious
    b. ranked via sentiment analysis

Chat for every trip or chat for friend group -> no subchats
Two different ways the ai can come in, one is chat specific, “lets’ run the lakers game next week” -> this will prompt a little submessage from the ai that is tagged to the message sent, users can fold it if theyre not interested. This includes things like ticket links, hotel booking options, eta information (“how long does it take to get from .. to ..”), music festivals happening soon, etc.
The second way the ai can come in is as a little pinned message reminding users of events that have been talked about, as a notification to the users to get people to hang out. “Remember the lakers game that you guys talked about? That has passed… but you are all free from … to …, and there’s … that’s happening. Want me to book it for you?”)

DEMO SCRIPT

The Demo: "Plans" (working title)

Setup on screen before you start: Group chat open, 4 members visible ("Maya, Jordan, Priya, you"), Trip Name: "LA Friends," expiry countdown in header shows **"6 days left"**, last message timestamp shows 3 weeks ago. Have the browser and the 4 Google Calendars pre-authed and ready to show.

[0:00–0:15] The hook

Presenter: "Group chats are where plans go to die. Four friends, a thread full of 'we should hang out soon' — and then three weeks of silence. We built Plans: a group chat that expires unless you actually use it, with an agent that makes sure you do."

On screen: camera on the chat. Expiry badge "6 days left" is visible. Presenter points to it.

[0:15–0:35] Reactive mode — the "folded" AI

Presenter: "Here's how it works during a normal conversation."

Driver types as Maya: "anyone down for the Lakers game next week?"

Agent reply appears threaded below, collapsed by default, with a small icon + "Plans found 3 options — tap to expand."

Driver taps to expand. Shows: 3 ticket options with prices, ETA from each person's location, a "propose to group" button.

Presenter: "The agent chimes in under the message — folded, so it's not in the way. Tickets, travel times, one tap to propose it to the group. It stays quiet unless someone actually mentions something real."

Driver collapses it back. Moves on.

[0:35–0:55] The silence + the proactive agent

Presenter: "But the real problem isn't when people are talking. It's when they're not."

Driver scrolls up — chat is empty for weeks. Back to the expiry badge: "6 days left."

A new agent message appears at the top, pinned, not threaded:

> **Plans** · *just now* Hey — this chat expires in 6 days. You four have all been quiet, but you're all free **Saturday 2–6pm**. There's a **free gallery opening in the Arts District at 3pm** — Maya's into contemporary art, Jordan's close by. Want me to set it up? **[Book it] [Show me something else] [Not this weekend]**

Presenter: "The agent read everyone's calendar, found the overlap, matched it to an actual event happening Saturday, and drafted the plan. No one had to ask."

[0:55–1:25] The one tap — the executable outcome

Presenter: "Here's the part that matters."

*Driver taps **Book it**.*

Screen shows a quick animation / status:

- ✅ RSVP'd for 4
- ✅ Added to Maya's calendar
- ✅ Added to Jordan's calendar
- ✅ Added to Priya's calendar
- ✅ Added to your calendar

Driver switches tabs to Google Calendar. Shows the event live on Saturday at 3pm. Switches to a second calendar — same event. Switches back to the chat.

In the chat, the agent posts a confirmation:

> **Plans** · *just now* Done. Saturday 3pm, Arts District Gallery. I'll remind you Friday night. Expiry reset — you've got 30 more days.

Presenter: "Four calendars updated. RSVP sent. And because you actually made a plan, the chat resets — it's earned another month."

[1:25–1:50] The pitch

Presenter: "Under the hood, this is five agents on Fetch.ai's Agentverse, coordinating via the Chat Protocol — calendar, interests, events, proposals, and booking. Discoverable via ASI:One.

Each one does one thing, and together they turn 'we should hang out' into a real event on a real calendar."

On screen: quick diagram of the 5 agents, Agentverse logo, ASI:One logo.

Presenter: "That's the Flicker to Flow thesis — turning coordination friction into an executable outcome. And it's the Fetch.ai thesis — user intent, real action, no hand-holding."

[1:50–2:00] The close

Presenter: "Group chats shouldn't be graveyards. With Plans, either you hang out, or the chat doesn't stick around. That's the deal."

On screen: tagline — ***"Plans. Or it's gone."***

Driver shows the Devpost/landing page / GitHub link.

Presenter: "Thanks."

Timing reality check

Segment	Target	Risk
Hook	15s	Low — rehearsable
Reactive demo	20s	**Medium** — live typing is fragile; pre-script the message
Silence + proactive	20s	Low — agent post is pre-triggered
One-tap booking	30s	**High** — Google Calendar API can lag; have a video fallback
Architecture pitch	25s	Low
Close	10s	Low
Total	**120s**	

Critical notes

The risky moment is the Google Calendar write. If the API hangs for 8 seconds, your demo dies. **Record a flawless run as a backup video** by hour 30. If live fails, cut to video mid-sentence. Practice the transition.

Every click in this demo needs to be scripted and rehearsed at least 10 times. The driver should be able to do it blindfolded by hour 34.

What's NOT in this demo (and shouldn't be): account creation, profile setup, birthdays, right-side panels, sentiment ranking, reactive-mode book-through, trip metadata beyond the name, manual calendar entry, paid-event checkout flow, substitute-event logic for missed plans. If a teammate suggests adding any of these "just for the demo," the answer is no.

What you need to build to make this demo work, in priority order:

1. The chat UI with 4 hardcoded users and a pre-seeded silent group
2. Calendar agent (free/busy across 4 real Google accounts)
3. Event agent (curated JSON, matches by tag)
4. Proposer agent (composes the proactive message)
5. Booking agent (writes to all 4 calendars, posts confirmation)
6. Expiry counter + reset-on-booking logic (visible in header, easy)
7. Reactive mode (folded reply under a specific message) — **build this last, cut if behind**

Build 1–6 first. Reactive mode is a bonus that makes the demo richer but isn't load-bearing. If hour 24 hits and you're still fighting Calendar OAuth, cut reactive without a second thought.

Want me to draft the Proposer agent's message template next, or the event JSON schema?

AI Agents

ASI:One and Agentverse

- ASI:One - chat platform like ChatGPT, based on user’s query will search for the best agent in Agentverse to answer that query
- Agentverse - a hub of AI agents, you can create and publish your own

Agentverse - Search and Discovery of Agents

- Fetch.ai enables autonomous, goal-oriented AI agents to discover, coordinate, and transact on behalf of users, businesses, and devices across an open, decentralized digital ecosystem. At LA Hacks, fetch.ai encourages participants to build on Agentverse, discoverable via ASI:One that turn user intent into real, executable outcomes.
- Develop a single or multi-agent orchestration that demonstrates reasoning, tool execution, and solves a real-world problem. Use any agentic framework like Claude

SDK, OpenAI Agent SDK, Google ADK, Langgraph, CrewAI, etc. of your choice OR simple plain python to bring your idea to life. Register your agents with Agentverse and implement the Chat Protocol (mandatory) & Payment Protocol (optional) to support direct ASI:One interactions and built-in monetization.

Build and Register AI Agents on Agentverse, discoverable via ASI:One, that turn user intent into real, executable outcomes.

Requirements to be eligible for a prize:

- Develop a single or multi-agent orchestration that demonstrates reasoning, tool execution, and solves a real-world problem.
- Use any agentic framework like Claude SDK, OpenAI Agent SDK, Google ADK, Langgraph, CrewAI, etc. of your choice OR simple plain python to bring your idea to life.
- Register your agents with Agentverse and implement the Chat Protocol (mandatory) & Payment Protocol (optional) to support direct ASI:One interactions and built-in monetization.
- No custom frontend is required - the use case must be demonstrated directly through ASI:One

Deliverables:

1. Share your ASI:One Chat session URL
   Example : https://asi1.ai/shared-chat/6d7caa05-94ff-4d33-8fa1-8e24d8ec032e
2. Agent(s) URL on Agentverse

Example:
https://agentverse.ai/agents/details/agent1qtuuyttz8ujuxceq0gllcerlksjneenrh2mfcm67st8qrm9lzzh3cd7f9h6/profile

3. Github Repo URL (Public) + demo video on Devpost

Agent Building Stack
Python
uAgents
Fetch Chat Protocol
LangGraph
OpenAI API
Google Calendar API
Pydantic
events.json

1 Agentverse agent
+ internal LangGraph multi-agent orchestration

```
plans-agent/
  agentverse/
    main.py              # uAgent + Chat Protocol wrapper
  graph/
    workflow.py          # LangGraph graph
    state.py             # shared state schema
    nodes/
      calendar_node.py
      interest_node.py
      event_node.py
      proposal_node.py
      booking_node.py
  data/
    events.json
    users.json
  services/
    google_calendar.py
    openai_client.py
```

Backend

Frontend